

Память CUDA

RAM память

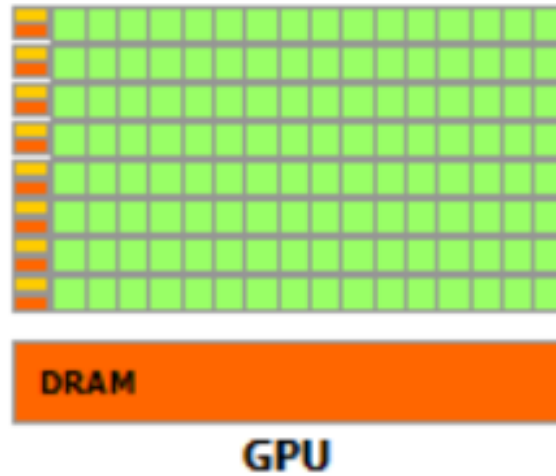
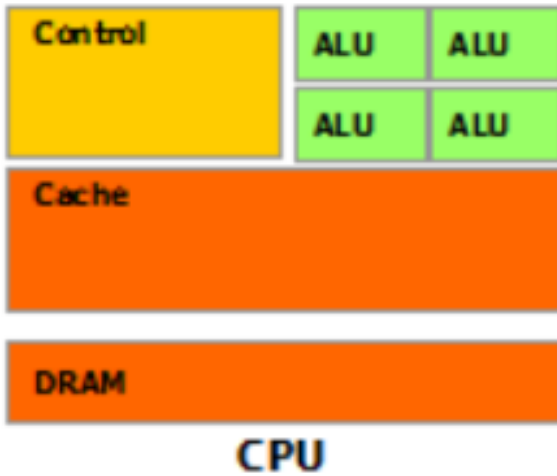
- Тип
- Объём
- Частота работы
- Тайминг
- Пропускная способность

DDR vs GDDR

- Graphics Double Data Rate
- Более высокая частота работы
- Более низкое энергопотребление
- Специальное управление буфером ввода/вывода

Отличия от CPU

- В CPU значимую часть занимают кэши различных уровней
- В CPU обычно не предоставляется прямой доступ к управлению кэшами



Виды памяти

- Регистры
- Разделяемая
- Локальная
- Глобальная
- Константная
- Текстурная
- Surface
- CPU RAM

Глобальная память

- Расположена на плате GPU
- Может быть выделена с CPU (CUDA_API) или с нитей
- Доступ для чтения / записи для всех нитей
- Высокая латентность
- Кэшируется начиная с Fermi

Регистровая память

- Регистры распределяются между блоками на этапе компиляции
- У каждой нити монопольный доступ к нескольким регистрам на все время исполнения ядра
- Доступ к регистрам других нитей запрещен
- Минимальная латентность

Локальная память

- Размещена в DRAM
- Время доступа порядка 400-800 тактов
- Хранит union-ы, массивы размеры которых неизвестны в момент компиляции

Разделяемая память

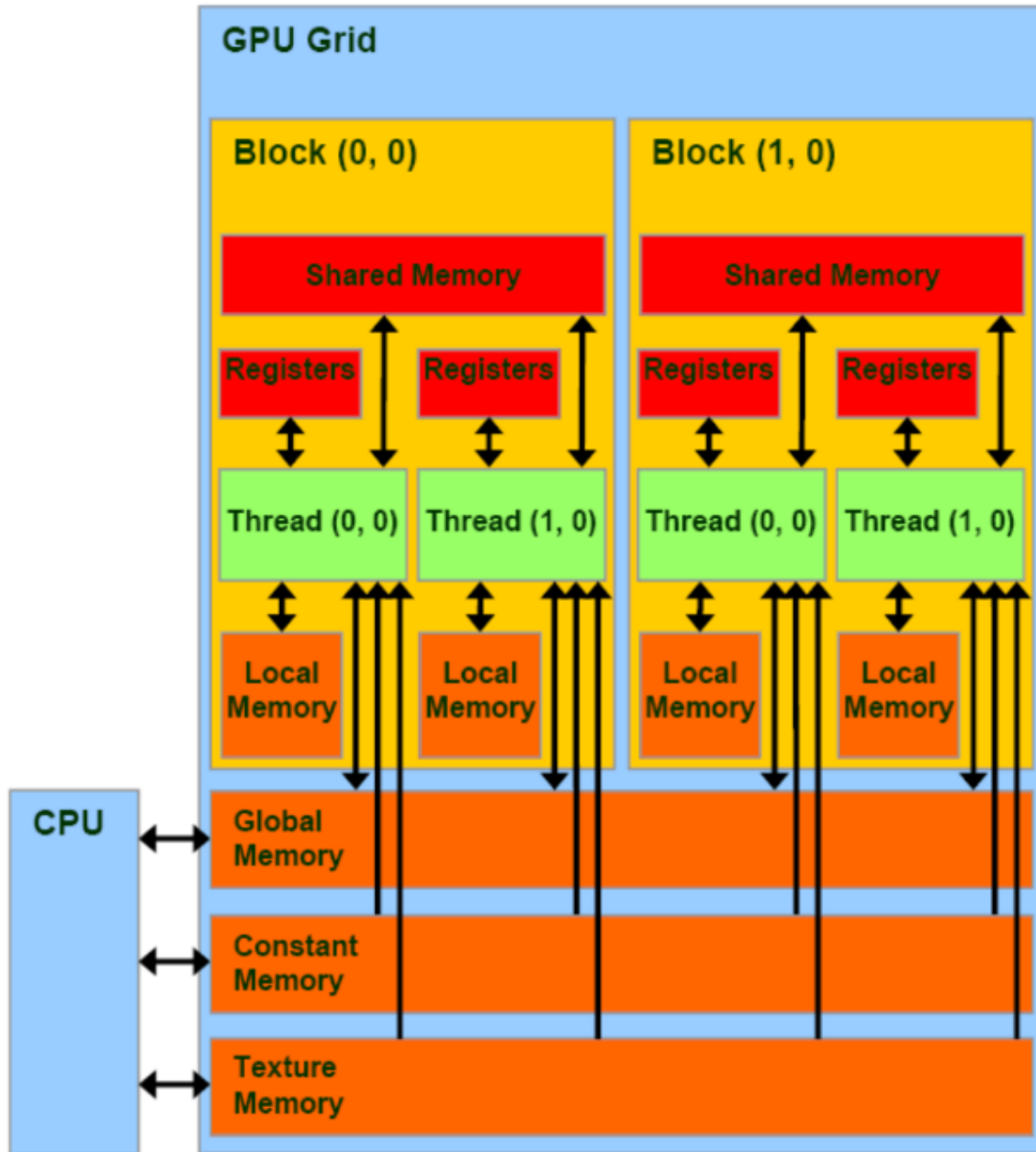
- Расположена в мультипроцессоре
- Выделяется на уровне блоков
- Общее ограничение по объему
- Малое время доступа
- Может использоваться всеми нитями блока для чтения и записи

Константная и текстурная памяти

- Расположены в DRAM
- Независимый кэш
- Относительно высокая скорость доступа
- Доступ сразу всем нитям только на чтение
- Общий объем ограничен

Виды памяти

- Регистры
- Разделяемая
- Локальная
- Глобальная
- Константная
- Текстурная
- Surface
- CPU RAM



Регистровая память

Характеристика	Значение
Расположение	мультипроцессор
Доступ	GPU - R/W, CPU - нет
Скорость доступа	максимальный
Время жизни	thread

- Не управляется из программного кода
- Размещение данных в регистрах выполняет компилятор

Локальная память

- Не управляется из программного кода
- Может быть использована компилятором при большом количестве локальных переменных в функции
- Значительно медленнее, чем регистровая
- Не предусмотрено явных средств, позволяющих блокировать использование локальной памяти

Локальная память

Характеристика	Значение
Расположение	DRAM
Кэшируемость	нет
Доступ	GPU - R/W, CPU - нет
Скорость доступа	низкая
Время жизни	thread

- Проанализировать код
- Исключить лишние локальные переменные

Константная память

```
cudaError_t cudaMemcpyToSymbol(const char * symbol,  
    const void * src, size_t count, size_t offset, enum  
    cudaMemcpyKind kind);
```

Кэшируемая область DRAM фиксированного размера

```
cudaError_t cudaMemcpyFromSymbol(void * dst, const char *  
    symbol, size_t count, size_t offset, enum cudaMemcpyKind  
    kind);
```

Доступная с GPU только для чтения

```
cudaError_t cudaMemcpyToSymbolAsync(const char *  
    symbol, const void * src, size_t count, size_t offset, enum  
    cudaMemcpyKind kind, cudaStream_t stream);
```

Для чтения с хоста

```
cudaError_t cudaMemcpyFromSymbolAsync(void * dst,  
    const char * symbol, size_t count, size_t offset, enum  
    cudaMemcpyKind kind, cudaStream_t stream);
```

Для записи с хоста

Константная память

Константная память выделяется непосредственно в коде программы при помощи спецификатора `__constant__`

```
__constant__ float constData [256]; // константная  
память GPU
```

```
float hostData [256]; // данные в памяти CPU
```

```
...
```

```
// Скопировать данные из памяти CPU в  
константную память GPU
```

```
cudaMemcpyToSymbol(constData, hostData,  
sizeof(data), 0, cudaMemcpyHostToDevice);
```

```
__global__ void kernel (float* pos)
```

```
{  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    pos[index] = pos[index] * constData[idx];  
}
```

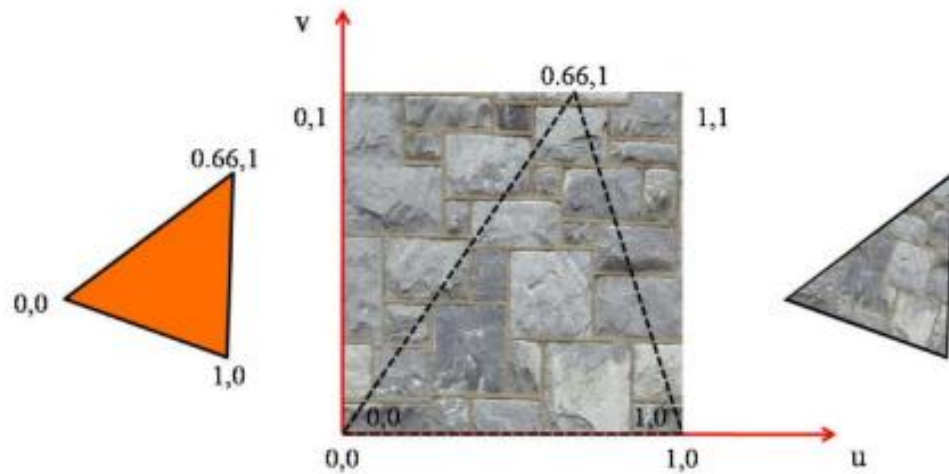
Константная память

Характеристика	Значение
Расположение	DRAM
Кэшируемость	да
Доступ	GPU - R/O, CPU - R/W
Скорость доступа	высокая(cache) / низкая
Время жизни	выделяется CPU

Подходит для размещения небольшого объема часто используемых неизменяемых данных, которые должны быть доступны всем нитям.

Текстурная память

Текстурная память +
аппаратные схемы
интерполяции =
текстурные блоки



Количество текстурных блоков зависит от архитектуры, используются в графических задачах для заполнения треугольников двумерными изображениями (текстурами)

Текстурная память

В текстурном блоке аппаратно реализованы функции:

- Фильтрация текстурных координат
- Билинейная или точечная интерполяция
- Разумное возвращаемое значение в случае, когда значения текстурных координат выходят за допустимые границы
- Обращение по нормализованным или целочисленным координатам
- Возвращение нормализованных значений
- Кэширование данных

Текстурная память

Предназначена главным образом для работы с текстурами и имеет специфические особенности в адресации, чтении и записи данных.

Выделяется с помощью `cudaMallocArray`

```
cudaError_t cudaMallocArray(struct cudaArray **arrayPtr,  
const struct cudaChannelFormatDesc *desc, size_t width,  
size_t height);
```

```
cudaError_t cudaFreeArray(struct cudaArray *array);
```

Текстурная память

ArrayPtr — не простой указатель, а непрозрачный контейнер

Доступ можно получить через «текстурные ссылки»

Задача абстракции — отделить данные и способ их хранения
от интерфейса доступа к ним

Для чтения из ядра сначала ассоциировать с его текстурной
ссылкой `cudaBindTextureToArray`

Чтение текстуры производится с помощью `tex1D`, `tex2D`,
`tex3D`

Глобальная память

- Максимальный доступный объем
- Произвольный доступ
- Сохраняет целостность данных на протяжении всего времени жизни приложения
- Основное хранилище для передачи данных между ядрами

Глобальная память

```
cudaError_t cudaMalloc(void ** devPtr, size_t size);
```

```
cudaError_t cudaFree(void * devPtr);
```

```
cudaError_t cudaMallocPitch(void ** devPtr, size_t *  
    pitch, size_t width, size_t height);
```

```
cudaError_t cudaMemcpy (void * dst, const void *  
    src, size_t size, enum cudaMemcpyKind kind );
```

```
cudaError_t cudaMemcpyAsync ( void * dst, const  
    void * src, size_t size, enum cudaMemcpyKind  
    kind, cudaStream_t stream );
```

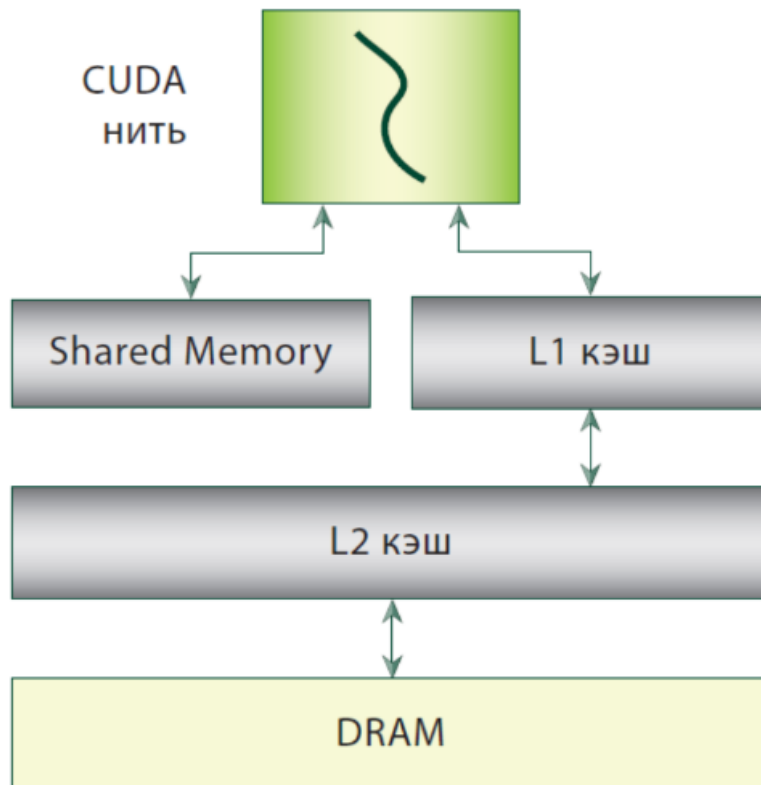

Глобальная память

- Прямая адресация (адресная арифметика)
- Указатель в контексте GPU
- Копирование данных
 - внутри устройства **device < = > device**
144 Гбайт / сек (Tesla C2050)
 - между хостом и устройством **host < = > device**
PCI-E 2.0 до 8 Гбайт/сек, на практике не более 4 Гбайт/сек

Глобальная память

Двухуровневое кэширование
начиная с Fermi

L2 = 768 Кбайт L1 + Shared =
16 + 48 (default)
48 + 16



Глобальная память

Транспонирование матриц

```
__global__ void transpose ( float * inData, float * outData, int n )
{
    unsigned int xIndex = blockDim.x * blockIdx.x + threadIdx.x;
    unsigned int yIndex = blockDim.y * blockIdx.y + threadIdx.y;
    unsigned int inIndex = xIndex + n * yIndex;
    unsigned int outIndex = yIndex + n * xIndex;
    outData [outIndex] = inData [inIndex];
}
```

Глобальная память

Характеристика	Значение
Расположение	DRAM
Кэшируемость	да, но не стоит надеяться
Доступ	GPU - R/W, CPU - R/W
Скорость доступа	низкая
Время жизни	выделяется CPU

- Передача данных между CPU и GPU
- Хранение данных во время работы программы

Разделяемая память

В текстурном блоке аппаратно реализованы функции:

- Расположена в мультипроцессоре
- Доступна всем нитям в одном блоке
- 16/48 Кбайт (compute capability 2.x)
- Объем делится поровну между всеми блоками, запущенными на мультипроцессоре

Разделяемая память

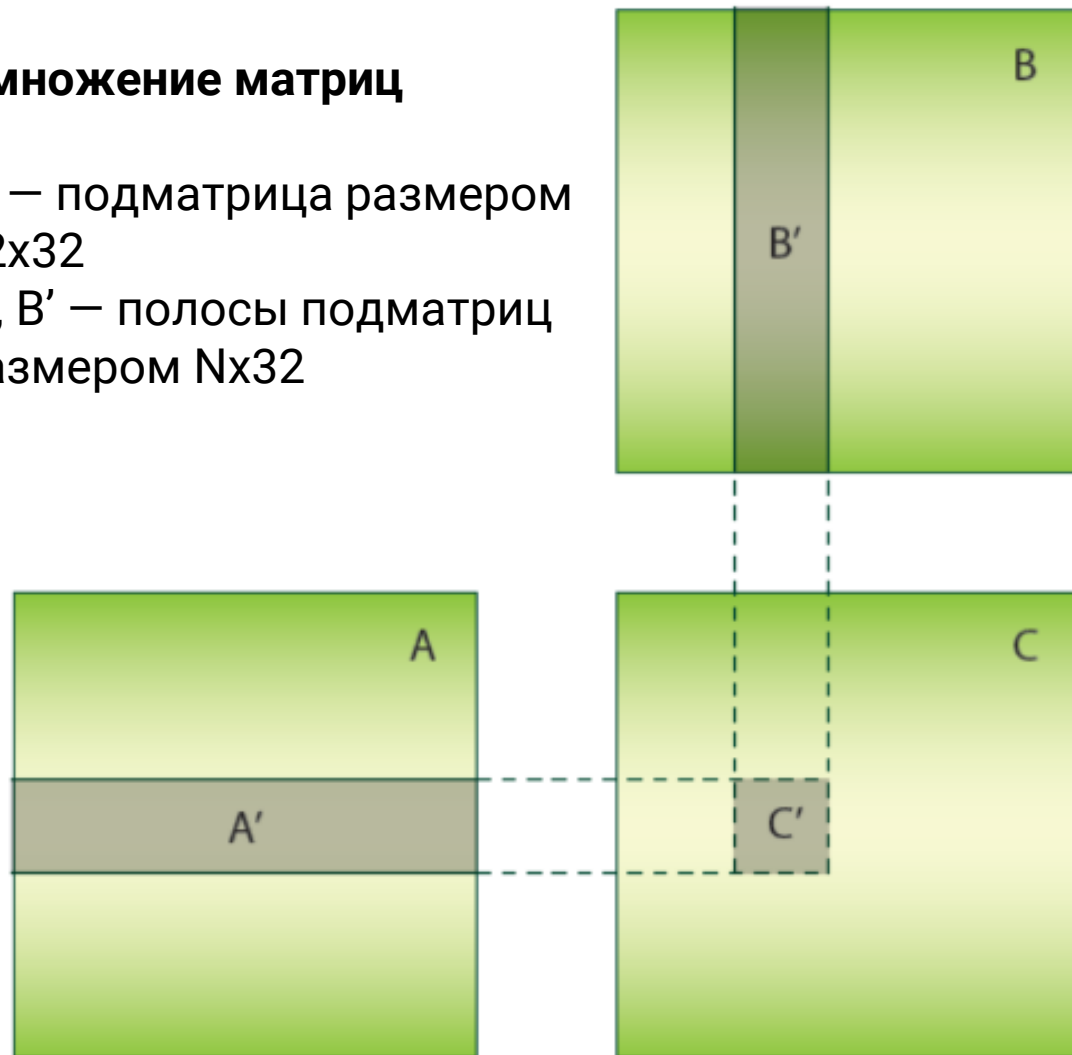
```
__global__ void kernel1(float* a)
{
    // Явно задано выделить 256*4 байт на блок.
    __shared__ float buf [256];
    // Запись значения из глобальной памяти в разделяемую.
    buf[threadIdx.x] = a[blockIdx.x * blockDim.x + threadIdx.x];
    ...
    __syncthreads ()
    ...
    // Запись результата обратно в глобальную память
}
```

Разделяемая память

Умножение матриц

C' — подматрица размером 32×32

A' , B' — полосы подматриц
размером $N \times 32$

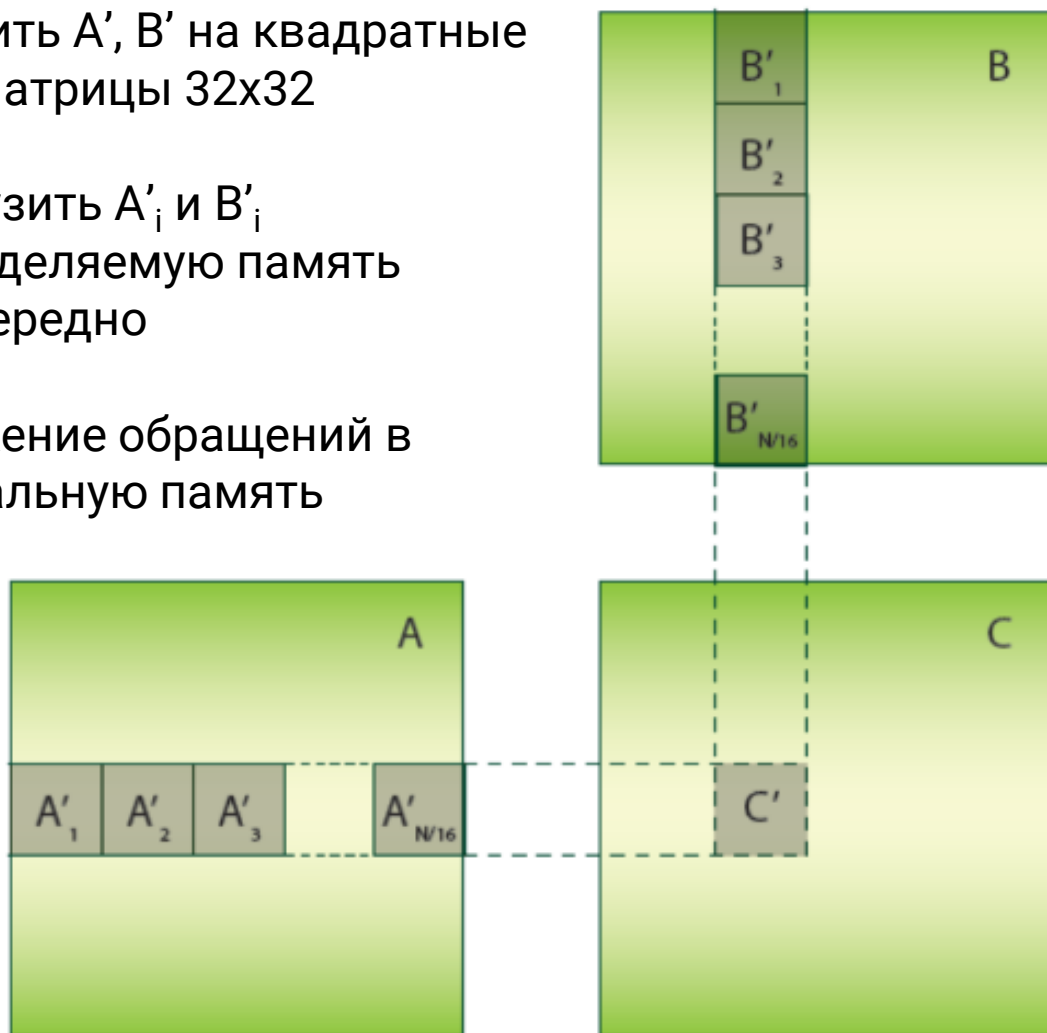


Разделяемая память

Разбить A' , B' на квадратные
Подматрицы 32×32

Загрузить A'_i и B'_i
в разделяемую память
поочередно

Снижение обращений в
глобальную память



Разделяемая память

Перемножение матриц 2048x2048 на Tesla C2070

Без использования разделяемой памяти
количество чтений из глобальной
памяти: 38 535 168
время выполнения: 324.63 мс

С использованием разделяемой памяти
количество чтений из глобальной
памяти: 1 196 032
время выполнения: 93.26 мс

Разделяемая память

Характеристика	Значение
Расположение	мультипроцессор
Кэшируемость	нет
Доступ	GPU - R/W, CPU - нет
Скорость доступа	высокая
Время жизни	block

Используется в качестве управляемого кэша первого уровня

Виды памяти

- Регистры
- Разделяемая
- Локальная
- Глобальная
- Константная
- Текстурная
- Surface
- CPU RAM

Память CUDA